Universidad Nacional Experimental de Guayana

Vicerrectorado Académico

P.F.G: Ingeniería en Informática

Unidad Curricular: Técnicas De Programación III

Sección: 3

Motor de búsqueda de rutas en mapas reales usando datos de OpenStreetMap.

Profesora:

Dubraska Roca

Integrantes:

Kenderth Muñoz 31782889

Introducción

El presente proyecto consiste en el desarrollo de un Sistema de Información Geográfica (SIG) simplificado diseñado para la planificación de rutas óptimas. La aplicación integra el uso de datos geográficos reales provenientes de OpenStreetMap (OSM) con algoritmos fundamentales de la teoría de grafos.

A diferencia de un mapa estático, este sistema modela la red vial como una estructura de datos dinámica (un grafo), donde las intersecciones y puntos de interés son representados como nodos y las calles como aristas con pesos asociados (distancias). El núcleo del software es el Algoritmo de Dijkstra, el cual permite calcular con precisión matemática el camino más corto entre dos coordenadas seleccionadas por el usuario a través de una interfaz gráfica interactiva construida en Java Swing.

En el desarrollo de aplicaciones de logística y navegación, el desafío no es solo mostrar un mapa, sino interpretar la vasta cantidad de datos geográficos para tomar decisiones eficientes. Este proyecto aborda tres problemáticas principales:

A. Abstracción de Datos Geográficos

El primer reto es cómo convertir coordenadas de latitud y longitud en una red lógica. El proyecto plantea la creación de un motor que tome puntos dispersos en un mapa y los vincule mediante una estructura de Lista de Adyacencia, permitiendo que el ordenador "entienda" qué calles están conectadas entre sí.

B. Optimización de la Ruta

Dada una red vial, existen infinitas combinaciones de caminos para llegar de un punto A a un punto B. El planteamiento se centra en la implementación de una Cola de Prioridad y el algoritmo de Dijkstra para reducir la complejidad computacional, garantizando que el sistema encuentre siempre la ruta con la distancia mínima en el menor tiempo posible.

C. Interacción Humano-Computador (HCI)

Un algoritmo por sí solo es difícil de usar para un usuario final. Por ello, el proyecto trata la integración de la librería JMapViewer para ofrecer una experiencia visual donde el usuario pueda:

-Visualizar cartografía real en tiempo real.

-Interactuar con el mapa mediante clics para definir puntos de interés.

-Observar la ruta resultante dibujada vectorialmente sobre las calles.

-Marco Teórico

-Teoría de Grafos: Definición rápida de nodos (vértices) y aristas (conexiones), y cómo representan el mundo real.

-Algoritmo de Dijkstra: Explicación de cómo funciona matemáticamente la búsqueda del camino más corto.

-Geocodificación: Concepto de traducción de nombres de lugares o coordenadas a puntos utilizables por el sistema.

-Objetivo General

Desarrollar una aplicación de escritorio en Java que permita la visualización de mapas interactivos y el cálculo de rutas óptimas entre dos puntos geográficos, utilizando el algoritmo de Dijkstra y datos de OpenStreetMap.

Objetivos Específicos

-Implementar una estructura de datos de Grafo: Diseñar y programar un modelo basado en listas de adyacencia que represente las intersecciones viales como nodos y las calles como aristas con pesos (distancias).

-Integrar el Algoritmo de Dijkstra: Desarrollar un motor de búsqueda eficiente que, mediante el uso de colas de prioridad, determine el camino más corto en el grafo construido.

-Gestionar Coordenadas Geográficas: Implementar un sistema de búsqueda de proximidad que permita traducir clics en un mapa (latitud y longitud) a los nodos lógicos más cercanos dentro de la base de datos del programa.

-Visualizar Cartografía Interactiva: Integrar la librería JMapViewer para renderizar mapas reales de OpenStreetMap, permitiendo al usuario navegar y seleccionar puntos de origen y destino de forma intuitiva.

-Representar Rutas Vectorialmente: Programar un componente visual capaz de dibujar el camino resultante del algoritmo directamente sobre el mapa, facilitando la interpretación de la ruta por parte del usuario.

-Metodología.

1. Representación del Espacio Físico (Modelo de Grafos)

-El mapa se abstrae utilizando una estructura de datos de Grafo.

-Las intersecciones o puntos de interés se modelan como vértices (Nodo Mapa), almacenando su latitud, longitud y un identificador de texto.

-Las calles que conectan estos puntos se modelan como Arista, las cuales guardan la distancia física (peso) y un multiplicador que simula la congestión vehicular (factorTrafico).

2. Traducción de Entradas del Usuario (Geocodificación Simulada)

-El sistema recoge la intención del usuario a través de una interfaz gráfica (nombres de calles o clics en el mapa).

-Para traducir estas entradas a nodos reales del grafo, la clase BuscadorNodoCercano utiliza dos métodos:

1-Por texto: Busca coincidencias entre la entrada del usuario y el nombre del nodo.

2-Por coordenadas (Clics): Aplica la fórmula de distancia euclidiana para encontrar el nodo matemático más próximo al punto seleccionado en la pantalla:

-Pruebas y Resultados

-Casos de Uso: Mostrar ejemplos prácticos de cómo responde el programa. Por ejemplo, documentar una búsqueda desde "Capitolio" hasta "Plaza Venezuela".

-Análisis de Variables: Mostrar cómo cambia la misma ruta si se selecciona "A pie" vs "Coche", o si se activa la casilla de "Evitar Tráfico".

-Capturas de Pantalla: Incluir imágenes de VentanaPrincipal mostrando la ruta dibujada y las direcciones generadas en el área de texto.

El proyecto busca demostrar que, mediante la combinación de programación orientada a objetos (POO) y algoritmos clásicos, es posible construir un motor de búsqueda de rutas funcional que resuelva problemas de movilidad urbana en un entorno controlado y visual.

Análisis Orientado a Objetos

a. Requerimientos Funcionales y No Funcionales

Requerimientos Funcionales:

-Selección de Puntos: El usuario debe poder establecer un punto de origen y un destino, ya sea escribiendo el nombre del lugar o haciendo clic en el mapa.

-Geocodificación y Búsqueda: El sistema debe buscar locaciones por coincidencia de texto o encontrando la coordenada más cercana al clic del usuario.

-Cálculo de Rutas: El sistema debe calcular el camino más corto entre el origen y el destino.

-Personalización de Ruta: El usuario debe poder seleccionar el medio de transporte (Coche, A pie, Bicicleta) y elegir si desea "Evitar Tráfico".

-Visualización: El sistema debe dibujar la ruta calculada sobre un mapa interactivo visual.

-Instrucciones Textuales: El sistema debe generar indicaciones paso a paso indicando los nodos a recorrer.

-Limpieza: El usuario debe poder borrar la ruta, los marcadores y los campos de texto.

Requerimientos No Funcionales:

-Usabilidad: Debe contar con una interfaz gráfica interactiva (GUI).

-Extensibilidad: El cálculo de rutas debe permitir la incorporación de futuros algoritmos sin alterar la lógica base.

-Dependencia: El sistema requiere la librería JMapViewer para la renderización.

c. Relaciones entre las Clases

-Herencia (Generalización): * Dijkstra es un BuscadorRuta. Hereda sus atributos y debe implementar su método abstracto.

-Composición (Fuerte): * VentanaPrincipal está compuesta por un MapaPanel. Si la ventana se destruye, el panel del mapa también.

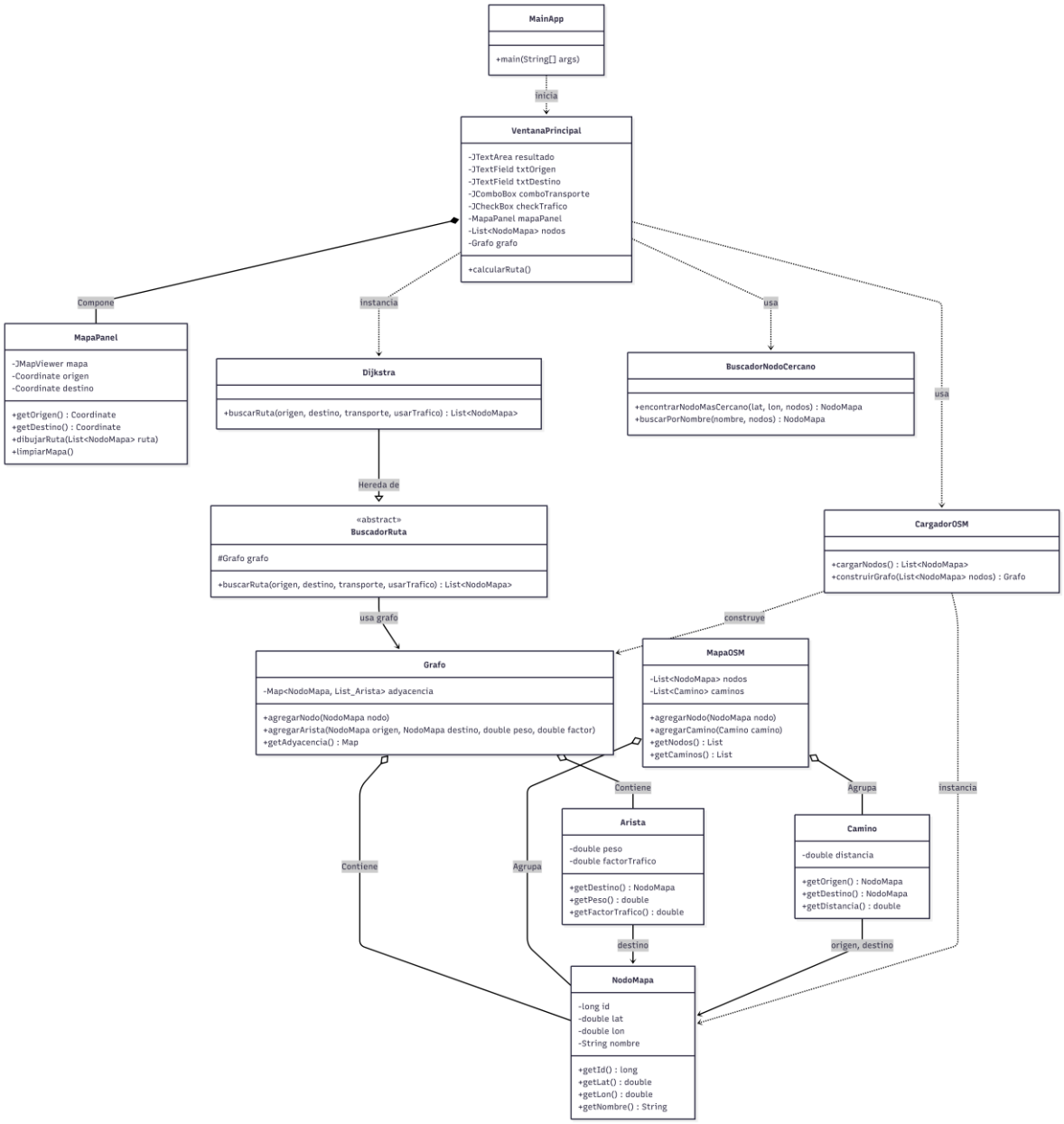
-Agregación (Débil): * Un Grafo tiene NodoMapa y Arista, pero los nodos existen independientemente del grafo.

-Un MapaOSM agrupa listas de NodoMapa y Camino.

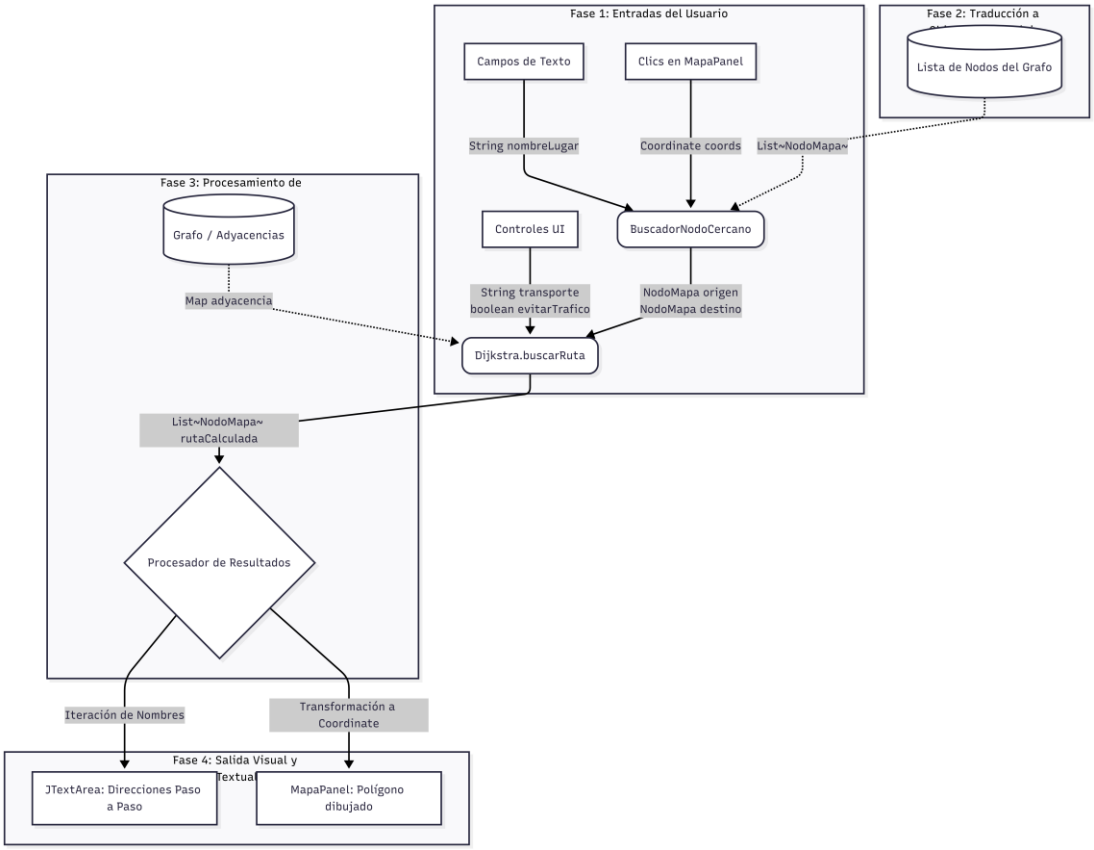
-Asociación (Uso): * BuscadorRuta conoce y utiliza el Grafo para realizar sus cálculos.

-VentanaPrincipal utiliza BuscadorNodoCercano y CargadorOSM como servicios.

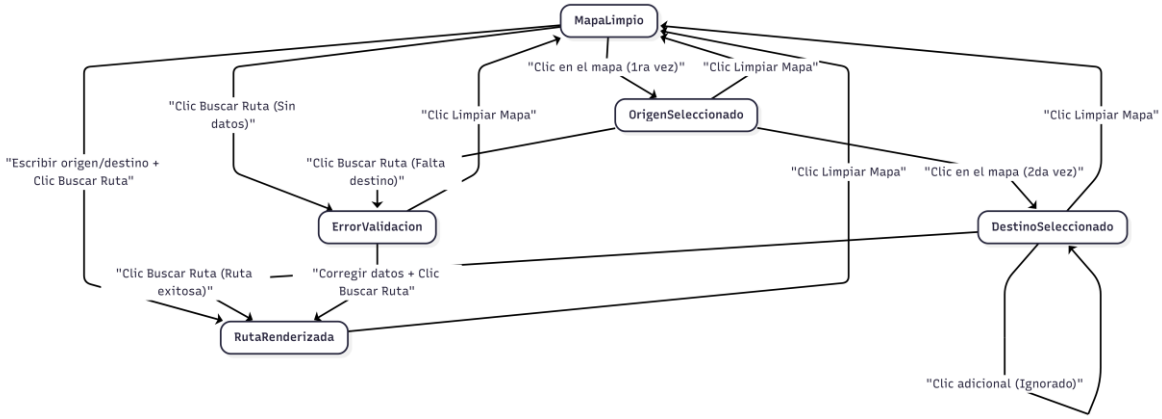
b. Diagrama de Clases.



d. Diagrama de Objetos



e.Diagrama de Transición de estados



f.Caso de Uso General.

Caso de Uso General: Calcular y Visualizar Ruta Óptima

Actor Principal: Usuario del sistema (peatón, conductor o ciclista).

-Propósito (Objetivo): Obtener el camino más eficiente entre dos puntos geográficos en un mapa interactivo, adaptado al medio de transporte y a las condiciones de tráfico.

-Precondiciones: La aplicación debe estar en ejecución y los datos del mapa (nodos y aristas) deben estar cargados correctamente en memoria por el CargadorOSM.

Diseño Orientado a Objetos

a. Conversión de Clases Conceptuales en Clases de Software en Java

-El Espacio Geográfico: El concepto de un mapa interconectado se tradujo a la clase de software Grafo. Para manejar la red de conexiones eficientemente en memoria, se utilizó la estructura de datos HashMap nativa de Java (Map<NodoMapa, List<Arista>> adyacencia), permitiendo búsquedas rápidas de vecinos.

-Puntos Físicos (Intersecciones/Lugares): Se convirtieron en la clase NodoMapa. Las coordenadas geográficas reales se implementaron usando tipos primitivos de alta precisión double para lat y lon, y se asignó un String para el nombre del lugar.

-Calles/Vías: Se representaron mediante la clase Arista, que encapsula una referencia en memoria al NodoMapa de destino y utiliza valores double para representar magnitudes matemáticas como el peso (distancia) y el factorTrafico.

-Proceso de Búsqueda: La idea de "encontrar un camino" se materializó usando la clase Dijkstra, la cual implementa la clase PriorityQueue de Java para ordenar dinámicamente los nodos según su costo acumulado durante la evaluación matemática.

b. Explicación del Patrón Modelo-Vista-Controlador (MVC).

Modelo (Model): Representa los datos y las reglas matemáticas del sistema, siendo totalmente independiente de la interfaz gráfica. En el proyecto, el Modelo está conformado por los paquetes mapa (clases NodoMapa, Camino), grafo (Grafo, Arista) y algoritmo (Dijkstra).

-Vista (View): Es la representación visual con la que interactúa el usuario. Está implementada en el paquete ui mediante las clases VentanaPrincipal (que organiza los paneles, botones y cuadros de texto usando javax.swing) y MapaPanel (que maneja la renderización gráfica de los mosaicos y polígonos usando JMapViewer).

-Controlador (Controller): Actúa como intermediario. En este proyecto, el rol del controlador es asumido parcialmente por la clase VentanaPrincipal a través de sus ActionListeners (eventos de botones). Cuando el usuario presiona "Buscar Ruta", el controlador captura los datos de la Vista, invoca a los servicios (BuscadorNodoCercano y Dijkstra) para procesar el Modelo, y finalmente ordena a la Vista (MapaPanel y JTextArea) que se actualice con los resultados.

c. Patrones de Diseño Utilizados

-Patrón Estrategia (Strategy): Se evidencia en la creación de la clase abstracta BuscadorRuta y su implementación concreta Dijkstra.

-Justificación: Al definir el método abstracto buscarRuta(...) en una clase padre, el sistema permite que en el futuro se puedan programar nuevos algoritmos (por ejemplo, una clase AStar o BellmanFord) que hereden de BuscadorRuta. El resto del programa (la interfaz gráfica) no necesitará ser modificado, ya que interactuará con la abstracción y no con la implementación específica.

d. Persistencia, Cohesión, Acoplamiento y Reutilización

-Persistencia: Actualmente, el sistema opera con un modelo de datos en memoria (volátil). No existe persistencia a largo plazo (como una base de datos SQL o archivos locales). En su lugar, la clase CargadorOSM actúa como un Mock (simulador) que " inyecciona " datos predefinidos en el grafo al momento de arrancar la aplicación. Una evolución futura requeriría conectar el sistema a la API de OpenStreetMap para lograr persistencia real.

-Cohesión (Alta): El sistema exhibe una alta cohesión, ya que cada clase tiene una única responsabilidad bien definida. Por ejemplo, MapaPanel solo se encarga de dibujar el mapa y capturar clics, BuscadorNodoCercano solo realiza cálculos de distancia euclidiana y búsqueda de cadenas de texto, y Dijkstra se dedica exclusivamente a resolver operaciones de grafos.

-Acoplamiento (Bajo): Existe un bajo nivel de dependencia entre los módulos principales. Las clases de lógica de negocio (como Grafo o Dijkstra) no tienen ninguna importación de librerías gráficas (javax.swing o jmapviewer). El acoplamiento ocurre solo en una dirección: la interfaz de usuario conoce al modelo de dominio, pero el modelo de dominio ignora la existencia de la interfaz.

-Reutilización: Gracias al bajo acoplamiento y a la alta cohesión, el nivel de reutilización del código es excelente. Toda la lógica empaquetada en algoritmo, grafo y mapa podría extraerse tal cual y utilizarse en un proyecto completamente distinto (como una aplicación móvil en Android o un servidor backend web) sin requerir modificaciones en su código fuente.

Pantallas del Programa

-Algoritmo

Buscador de ruta

src > algoritmo > BuscadorRuta.java > BuscadorRuta

```java
package algoritmo;

import grafo.Grafo;
import mapa.NodoMapa;
import java.util.List;

public abstract class BuscadorRuta {
    protected Grafo grafo;

    public BuscadorRuta(Grafo grafo) { this.grafo = grafo; }

    // Nuevo: Considera el medio de transporte y si se debe evitar el tráfico
    public abstract List<NodoMapa> buscarRuta(NodoMapa origen, NodoMapa destino, String transporte, boolean usarTrafico);
}
```

Dijkstra

```java
package algoritmo;

import grafo.*;
import mapa.NodoMapa;
import java.util.*;

public class Dijkstra extends BuscadorRuta {

    public Dijkstra(Grafo grafo) { super(grafo); }

    @Override
    public List<NodoMapa> buscarRuta(NodoMapa origen, NodoMapa destino, String transporte, boolean usarTrafico) {
        Map<NodoMapa, Double> dist = new HashMap<>();
        Map<NodoMapa, NodoMapa> prev = new HashMap<>();
        PriorityQueue<NodoMapa> cola = new PriorityQueue<>(Comparator.comparingDouble(dist::get));

        for (NodoMapa n : grafo.getAdyacencia().keySet()) {
            dist.put(n, Double.MAX_VALUE);
        }

        dist.put(origen, value: 0.0);
        cola.add(origen);

        while (!cola.isEmpty()) {
            NodoMapa actual = cola.poll();
            if (actual.equals(destino)) break;

            for (Arista arista : grafo.getAdyacencia().get(actual)) {
                NodoMapa vecino = arista.getDestino();

                // Algoritmo de rutas adaptativo
                double multiplicadorTransporte = transporte.equals(anObject: "A pie") ? 0.8 : (transporte.equals(anObject: "Bicicleta") ? 0.9 : 1.
                double multiplicadorTrafico = (usarTrafico && transporte.equals(anObject: "Coche")) ? arista.getFactorTrafico() : 1.0;
```

Java: Ready Lín. 16, col. 1 Espacios: 4 UTF-8 CRLF {} Java

```java
                // Algoritmo de rutas adaptativo
                double multiplicadorTransporte = transporte.equals(anObject: "A pie") ? 0.8 : (transporte.equals(anObject: "Bicicleta") ? 0.9 : 1.
                double multiplicadorTrafico = (usarTrafico && transporte.equals(anObject: "Coche")) ? arista.getFactorTrafico() : 1.0;

                double costoArista = arista.getPeso() * multiplicadorTransporte * multiplicadorTrafico;
                double nuevaDist = dist.get(actual) + costoArista;

                if (nuevaDist < dist.get(vecino)) {
                    dist.put(vecino, nuevaDist);
                    prev.put(vecino, actual);
                    cola.add(vecino);
                }
            }
        }

        List<NodoMapa> ruta = new ArrayList<>();
        NodoMapa paso = destino;
        while (paso != null) {
            ruta.add(paso);
            paso = prev.get(paso);
        }
        Collections.reverse(ruta);

        // Si no hay ruta (el primer elemento no es el origen), devolver lista vacía
        if(ruta.size() > 0 && !ruta.get(index: 0).equals(origen)) return new ArrayList<>();

        return ruta;
    }
}
```

-Grafos

Aristas

```java
package grafo;

import mapa.NodoMapa;

public class Arista {
    private NodoMapa destino;
    private double peso;
    private double factorTrafico; // Nuevo: Simula congestión (1.0 = normal, 2.0 = mucho tráfico)

    public Arista(NodoMapa destino, double peso, double factorTrafico) {
        this.destino = destino;
        this.peso = peso;
        this.factorTrafico = factorTrafico;
    }

    public NodoMapa getDestino() { return destino; }
    public double getPeso() { return peso; }
    public double getFactorTrafico() { return factorTrafico; }
}
```

Grafos

```java
package grafo;

import mapa.NodoMapa;
import java.util.*;

public class Grafo {
    private Map<NodoMapa, List<Arista>> adyacencia = new HashMap<>();

    public void agregarNodo(NodoMapa nodo) {
        adyacencia.putIfAbsent(nodo, new ArrayList<>());
    }

    public void agregarArista(NodoMapa origen, NodoMapa destino, double peso, double factorTrafico) {
        adyacencia.get(origen).add(new Arista(destino, peso, factorTrafico));
    }

    public Map<NodoMapa, List<Arista>> getAdyacencia() { return adyacencia; }
}
```

-Main

```java
package main;

import ui.VentanaPrincipal;

public class MainApp {
    Run | Debug
    public static void main(String[] args) {
        new VentanaPrincipal().setVisible(b: true);
    }
}
```

-Mapa

Camino

```java
package mapa;

public class Camino {
    private NodoMapa origen;
    private NodoMapa destino;
    private double distancia;

    public Camino(NodoMapa origen, NodoMapa destino, double distancia) {
        this.origen = origen;
        this.destino = destino;
        this.distancia = distancia;
    }

    public NodoMapa getOrigen() { return origen; }
    public NodoMapa getDestino() { return destino; }
    public double getDistancia() { return distancia; }
}
```

Mapa

```java
package mapa;

import java.util.ArrayList;
import java.util.List;

public class MapaOSM {
    private List<NodoMapa> nodos = new ArrayList<>();
    private List<Camino> caminos = new ArrayList<>();

    public void agregarNodo(NodoMapa nodo) { nodos.add(nodo); }
    public void agregarCamino(Camino camino) { caminos.add(camino); }
    public List<NodoMapa> getNodos() { return nodos; }
    public List<Camino> getCaminos() { return caminos; }
}
```

NodoMapa

```java
package mapa;

public class NodoMapa {
    private long id;
    private double lat;
    private double lon;
    private String nombre; // Nuevo: Para geocodificación simulada

    public NodoMapa(long id, double lat, double lon, String nombre) {
        this.id = id;
        this.lat = lat;
        this.lon = lon;
        this.nombre = nombre;
    }

    public long getId() { return id; }
    public double getLat() { return lat; }
    public double getLon() { return lon; }
    public String getNombre() { return nombre; }
}
```

-Servicio

BuscardorNodo

```java
package servicio;

import mapa.NodoMapa;
import java.util.List;

public class BuscadorNodoCercano {

    public NodoMapa encontrarNodoMasCercano(double lat, double lon, List<NodoMapa> nodos) {
        NodoMapa mejor = null;
        double mejorDist = Double.MAX_VALUE;
        for (NodoMapa n : nodos) {
            double dx = lat - n.getLat();
            double dy = lon - n.getLon();
            double dist = Math.sqrt(dx*dx + dy*dy);
            if (dist < mejorDist) { mejorDist = dist; mejor = n; }
        }
        return mejor;
    }

    // Nuevo: Geocodificación simulada (Búsqueda por texto)
    public NodoMapa buscarPorNombre(String nombre, List<NodoMapa> nodos) {
        for (NodoMapa n : nodos) {
            if (n.getNombre().toLowerCase().contains(nombre.toLowerCase())) {
                return n;
            }
        }
        return null;
    }
}
```

CargadorOSM

```java
package servicio;
import mapa.NodoMapa;
import grafo.Grafo;
import java.util.*;
public class CargadorOSM {

    public List<NodoMapa> cargarNodos() {
        List<NodoMapa> nodos = new ArrayList<>();
        nodos.add(new NodoMapa(id: 1, lat: 10.48, -66.90, nombre: "Capitolio"));
        nodos.add(new NodoMapa(id: 2, lat: 10.47, -66.88, nombre: "La Hoyada"));
        nodos.add(new NodoMapa(id: 3, lat: 10.46, -66.86, nombre: "Bellas Artes"));
        nodos.add(new NodoMapa(id: 4, lat: 10.45, -66.84, nombre: "Plaza Venezuela"));
        nodos.add(new NodoMapa(id: 5, lat: 10.44, -66.82, nombre: "Sabana Grande"));
        return nodos;
    }
    public Grafo construirGrafo(List<NodoMapa> nodos) {
        Grafo g = new Grafo();
        for (NodoMapa n : nodos) { g.agregarNodo(n); }

        conectar(g, nodos.get(index: 0), nodos.get(index: 1), trafico: 2.5);
        conectar(g, nodos.get(index: 1), nodos.get(index: 2), trafico: 1.0);
        conectar(g, nodos.get(index: 2), nodos.get(index: 3), trafico: 3.0);
        conectar(g, nodos.get(index: 3), nodos.get(index: 4), trafico: 1.2);
        return g;
    }
    private void conectar(Grafo g, NodoMapa a, NodoMapa b, double trafico){
        double dx = a.getLat()-b.getLat();
        double dy = a.getLon()-b.getLon();
        double dist = Math.sqrt(dx*dx+dy*dy);
        g.agregarArista(a, b, dist, trafico);
        g.agregarArista(b, a, dist, trafico);
    }
}
```

-UI

MapaPanel

```java
package ui;

import org.openstreetmap.gui.jmapviewer.*;
import org.openstreetmap.gui.jmapviewer.interfaces.ICoordinate;

import javax.swing.*;
import java.awt.event.MouseAdapter;
import java.awt.event.MouseEvent;

public class MapaPanel extends JPanel {
    private JMapViewer mapa;
    private Coordinate origen;
    private Coordinate destino;

    public MapaPanel() {
        setLayout(new java.awt.BorderLayout());
        mapa = new JMapViewer();
        mapa.setDisplayPosition(new Coordinate(10.48,-66.90), 12);
        add(mapa);

        mapa.addMouseListener(new MouseAdapter() {
            public void mouseClicked(MouseEvent e) {
                ICoordinate ic = mapa.getPosition(e.getPoint());
                Coordinate c = new Coordinate(ic.getLat(),ic.getLon());

                if(origen==null){
                    origen=c;
                    mapa.addMapMarker(new MapMarkerDot("Origen",origen));
                }else if(destino==null){
                    destino=c;
                    mapa.addMapMarker(new MapMarkerDot("Destino",destino));
                }
            }
```

```java
                ICoordinate ic = mapa.getPosition(e.getPoint());
                Coordinate c = new Coordinate(ic.getLat(),ic.getLon());

                if(origen==null){
                    origen=c;
                    mapa.addMapMarker(new MapMarkerDot("Origen",origen));
                }else if(destino==null){
                    destino=c;
                    mapa.addMapMarker(new MapMarkerDot("Destino",destino));
                }
            }
        });
    }

    public Coordinate getOrigen(){ return origen; }
    public Coordinate getDestino(){ return destino; }

    public void dibujarRuta(java.util.List<mapa.NodoMapa> ruta){
        java.util.List<Coordinate> coords = new java.util.ArrayList<>();
        for(mapa.NodoMapa n : ruta){
            coords.add(new Coordinate(n.getLat(), n.getLon()));
        }
        mapa.addMapPolygon(new MapPolygonImpl(coords));
    }

    public void limpiarMapa() {
        mapa.removeAllMapPolygons();
        mapa.removeAllMapMarkers();
        origen = null;
        destino = null;
    }
}
```

VentanaPrincipal

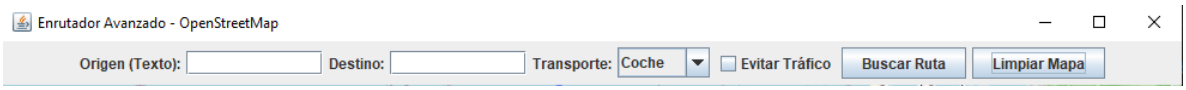
```java
package ui;

import algoritmo.Dijkstra;
import grafo.Grafo;
import mapa.NodoMapa;
import servicio.BuscadorNodoCercano;
import servicio.CargadorOSM;
import org.openstreetmap.gui.jmapviewer.Coordinate;

import javax.swing.*;
import java.awt.*;
import java.util.List;

public class VentanaPrincipal extends JFrame {

    private JTextArea resultado;
    private MapaPanel mapaPanel;
    private List<NodoMapa> nodos;
    private Grafo grafo;
    private JTextField txtOrigen;
    private JTextField txtDestino;
    private JComboBox<String> comboTransporte;
    private JCheckBox checkTrafico;

    public VentanaPrincipal(){
        setTitle(title: "Enrutador Avanzado - OpenStreetMap");
        setSize(width: 1000, height: 700);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setLayout(new BorderLayout());

        mapaPanel = new MapaPanel();
        add(mapaPanel, BorderLayout.CENTER);

```

```java
        setLayout(new BorderLayout());

        mapaPanel = new MapaPanel();
        add(mapaPanel, BorderLayout.CENTER);

        JPanel panelControles = new JPanel(new FlowLayout());
        txtOrigen = new JTextField(columns: 10);
        txtDestino = new JTextField(columns: 10);
        comboTransporte = new JComboBox<>(new String[]{"Coche", "A pie", "Bicicleta"});
        checkTrafico = new JCheckBox(text: "Evitar Tráfico");
        JButton buscar = new JButton(text: "Buscar Ruta");
        JButton limpiar = new JButton(text: "Limpiar Mapa");

        panelControles.add(new JLabel(text: "Origen (Texto):"));
        panelControles.add(txtOrigen);
        panelControles.add(new JLabel(text: "Destino:"));
        panelControles.add(txtDestino);
        panelControles.add(new JLabel(text: "Transporte:"));
        panelControles.add(comboTransporte);
        panelControles.add(checkTrafico);
        panelControles.add(buscar);
        panelControles.add(limpiar);
        add(panelControles, BorderLayout.NORTH);

        resultado = new JTextArea(rows: 8, columns: 20);
        resultado.setEditable(b: false);
        add(new JScrollPane(resultado), BorderLayout.SOUTH);

        CargadorOSM cargador = new CargadorOSM();
        nodos = cargador.cargarNodos();
        grafo = cargador.construirGrafo(nodos);
```

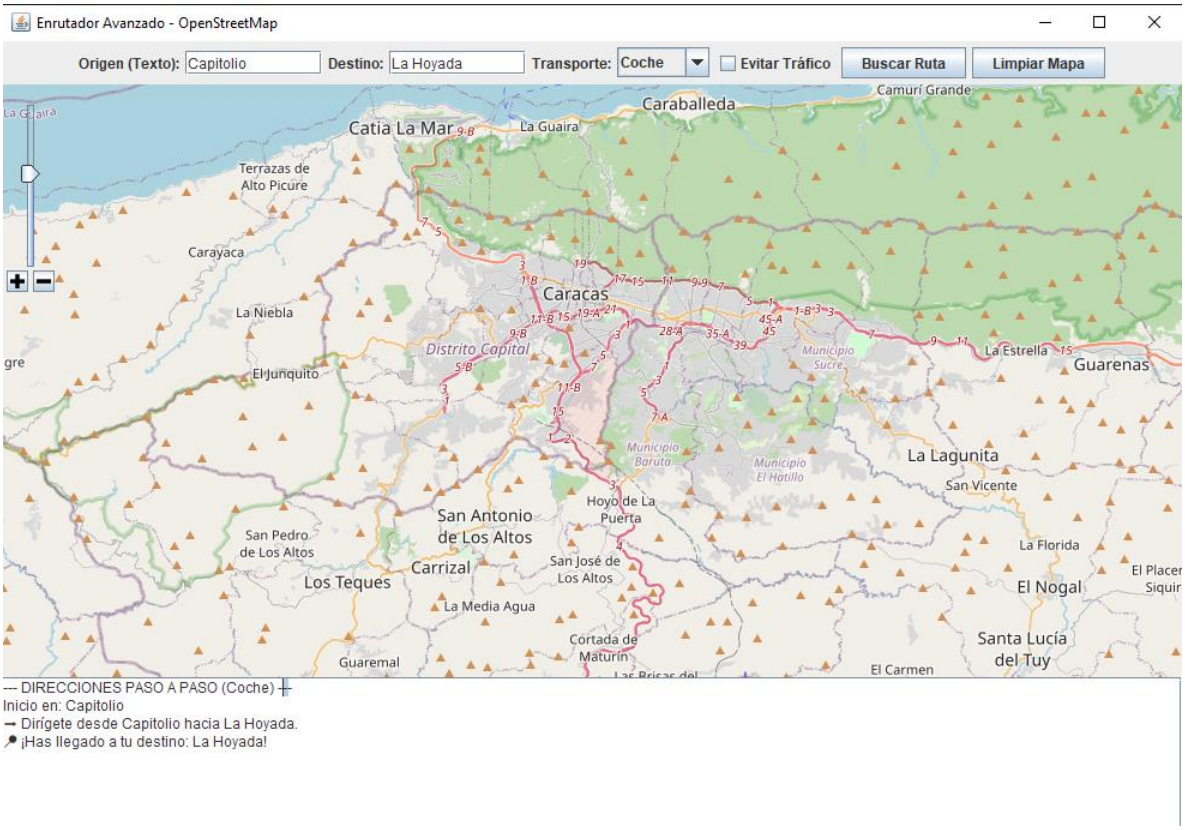
```java
        CargadorOSM cargador = new CargadorOSM();
        nodos = cargador.cargarNodos();
        grafo = cargador.construirGrafo(nodos);

        buscar.addActionListener(e -> calcularRuta());
        limpiar.addActionListener(e -> {
            mapaPanel.limpiarMapa();
            resultado.setText(t: "");
            txtOrigen.setText(t: "");
            txtDestino.setText(t: "");
        });
    }

    private void calcularRuta(){
        BuscadorNodoCercano buscador = new BuscadorNodoCercano();
        NodoMapa nodoOrigen = null;
        NodoMapa nodoDestino = null;

        if (!txtOrigen.getText().isEmpty() && !txtDestino.getText().isEmpty()) {
            nodoOrigen = buscador.buscarPorNombre(txtOrigen.getText(), nodos);
            nodoDestino = buscador.buscarPorNombre(txtDestino.getText(), nodos);
        }
        else {
            Coordinate cOrigen = mapaPanel.getOrigen();
            Coordinate cDestino = mapaPanel.getDestino();
            if(cOrigen != null && cDestino != null){
                nodoOrigen = buscador.encontrarNodoMasCercano(cOrigen.getLat(), cOrigen.getLon(), nodos);
                nodoDestino = buscador.encontrarNodoMasCercano(cDestino.getLat(), cDestino.getLon(), nodos);
            }
        }

```

```java
        }

        if(nodoOrigen == null || nodoDestino == null){
            resultado.setText(t: "Error: Seleccione en el mapa o escriba lugares válidos (ej. Capitolio, La Hoyada).");
            return;
        }

        String transporte = (String) comboTransporte.getSelectedItem();
        boolean considerarTrafico = checkTrafico.isSelected();

        Dijkstra dijkstra = new Dijkstra(grafo);
        List<NodoMapa> ruta = dijkstra.buscarRuta(nodoOrigen, nodoDestino, transporte, considerarTrafico);

        if (ruta.isEmpty()) {
            resultado.setText(t: "No se encontró una ruta.");
            return;
        }

        mapaPanel.limpiarMapa();
        mapaPanel.dibujarRuta(ruta);

        resultado.setText("--- DIRECCIONES PASO A PASO (" + transporte + ") ---\n");
        resultado.append("Inicio en: " + ruta.get(index: 0).getNombre() + "\n");

        for (int i = 0; i < ruta.size() - 1; i++) {
            NodoMapa actual = ruta.get(i);
            NodoMapa siguiente = ruta.get(i + 1);
            resultado.append("➡ Dirígete desde " + actual.getNombre() + " hacia " + siguiente.getNombre() + ".\n");
        }
        resultado.append("📍 ¡Has llegado a tu destino: " + ruta.get(ruta.size() - 1).getNombre() + "!");
```

-Botones de la interfaz



-Código Copilado



Conclusión

¿Qué se hizo?

Para construir el (Enrutador Avanzado), se aplicó una metodología rigurosa que partió de la comprensión del problema del mundo real hasta llegar a la implementación en código, dividida en dos grandes fases:

-En la fase de Análisis (OOA):

Se abstrajo el dominio del problema (la navegación geográfica) aislando sus componentes esenciales. Las calles, intersecciones y el tráfico del mundo real se modelaron conceptualmente utilizando la Teoría de Grafos.

Se definieron claramente los límites del sistema mediante la captura de Requerimientos Funcionales y No Funcionales, y se estructuró la interacción del usuario a través de Casos de Uso detallados.

Se modeló el comportamiento dinámico del sistema (cómo reacciona a los clics del usuario) mediante un Diagrama de Transición de Estados.

-En la fase de Diseño (OOD):

Se tradujeron los modelos conceptuales a una arquitectura de software real estructurada en paquetes de Java (algoritmo, grafo, mapa, servicio, ui).

Se adoptó el patrón arquitectónico Modelo-Vista-Controlador (MVC) para separar estrictamente la lógica matemática de la interfaz visual.

Se aplicaron principios de diseño sólido (como el uso de clases abstractas y el patrón Strategy en BuscadorRuta) para gestionar la complejidad matemática del algoritmo de Dijkstra.

¿Qué se logró?

La aplicación estricta del Análisis y Diseño Orientado a Objetos permitió alcanzar un sistema robusto, funcional y profesional. Los principales logros técnicos son:

-Alta Cohesión y Bajo Acoplamiento: Se logró que cada módulo haga solo una cosa y la haga bien. La lógica matemática (Dijkstra, Grafo) es completamente independiente de cómo se muestran los datos en la pantalla (VentanaPrincipal, MapaPanel). Esto significa que el código de la interfaz gráfica no interfiere con los cálculos de la ruta.

-Máxima Extensibilidad (Escalabilidad): Gracias al diseño de interfaces y clases abstractas, el sistema está preparado para crecer. Si el día de mañana se desea agregar un nuevo algoritmo de búsqueda (como A* o Bellman-Ford), solo hay que crear una clase que herede de BuscadorRuta sin tener que reescribir ni una sola línea de la interfaz gráfica de usuario.

-Reutilización de Código: Los módulos creados (grafo y algoritmo) se convirtieron en componentes genéricos. Podrían ser extraídos y reutilizados en una aplicación móvil, web o de consola sin sufrir alteraciones, demostrando un excelente diseño agnóstico a la plataforma.

-Resolución Eficaz del Problema: Finalmente, a nivel de producto, se logró una aplicación de escritorio plenamente interactiva que resuelve el problema de enrutamiento. El usuario recibe un feedback inmediato, traduciendo cálculos matemáticos complejos en información visual (un polígono trazado en el mapa) y textual (direcciones paso a paso comprensibles por humanos).

-Que se hizo y que se logró con relación a la solución presentada utilizando la Teoría de Grafos.

¿Qué se hizo?

Para poder calcular la ruta óptima, el entorno físico de la ciudad fue traducido a un modelo matemático puro utilizando los fundamentos de la Teoría de Grafos:

-Abstracción de Vértices (Nodos): Los puntos de interés y las intersecciones de las calles se modelaron como vértices, representados por la clase NodoMapa, encapsulando sus coordenadas espaciales exactas (latitud y longitud).

-Definición de Aristas (Conexiones): Las vías transitables se modelaron como aristas direccionales mediante la clase Arista. A cada arista se le asignó un "peso" numérico base (la distancia física) y un atributo adicional para simular restricciones del mundo real (factorTrafico).

-Estructura de Datos Óptima: Para representar el grafo completo en la memoria del sistema, se implementó una Lista de Adyacencia utilizando un mapa de tipo clave-valor (Map<NodoMapa, List<Arista>>) en la clase Grafo.

-Implementación del Algoritmo de Dijkstra: Se adaptó el clásico algoritmo de caminos mínimos matemáticos. Se configuró para recorrer el grafo evaluando los vecinos de cada nodo mediante una Cola de Prioridad (PriorityQueue), la cual ordena dinámicamente los nodos según su costo acumulado.

-Ponderación Dinámica: Se introdujo una innovación en la evaluación de los pesos: el algoritmo multiplica el peso base de la arista por modificadores matemáticos en tiempo real según el transporte seleccionado (ej. 0.8 para "A pie") y el nivel de congestión de la vía.

¿Qué se logró?

La implementación de este modelo matemático trajo consigo resultados altamente eficientes para el enrutamiento:

-Alta Eficiencia Espacial: Al utilizar una Lista de Adyacencia en lugar de una Matriz de Adyacencia, el sistema logró un almacenamiento en memoria altamente optimizado. Solo se almacenan las conexiones reales (calles existentes), lo que permite que el programa pueda escalar para soportar miles de nodos de OpenStreetMap sin agotar la memoria RAM.

-Toma de Decisiones Realistas: Se logró superar la limitación del "camino más corto tradicional". Al alterar los pesos de las aristas matemáticamente durante la ejecución, el grafo modela comportamientos del mundo real. El sistema es capaz de desviar al usuario por una ruta físicamente más larga si la ruta corta tiene un factorTrafico elevado.

-Garantía Matemática de Optimización: El uso de la estructura del Grafo combinada con el algoritmo de Dijkstra asegura que el sistema no adivina la ruta, sino que explora sistemáticamente

las ramificaciones hasta encontrar y demostrar matemáticamente el camino de menor costo posible (o determinar con exactitud si un destino es inalcanzable).